Robotik Mühendisliği Serisi — Cilt 0

Yol Haritası

Robotik mühendisliğine giden yolun planı

Türkçe–İngilizce karma konu anlatımı
teori · sayısal örnekler · çalışan Python ve C++ kodları

Odak: bacaklı ve humanoid robotlar

31 Ekim 2026

İçindekiler

1 Bu Seri Nedir, Nasıl Çalışılır **1**

1.1 Hedef: “tam teşekküllü robotik mühendisi” ne demek? 1

1.2 Her cildin yapısı 1

1.3 Nasıl çalışmalısın 2

1.4 Araç kurulumu (kendi bilgisayarında) 2

2 Ciltler ve Yol Haritası **3**

2.1 Yedi cilt 3

2.2 Sıra ve bağımlılıklar 4

2.3 Zaman çizelgesi (haftada 10–15 saat) 4

3 Proje Merdiveni **5**

3.1 Seri boyunca kullanılacak robot: SB-1 5

3.2 P2 hakkında: MG996R ile neyi yapabilir, neyi yapamazsın? 6

3.3 Portföy ve başvuru için 6

4 Beceri Kontrol Listesi **7**

4.1 Matematik ve mekanik (Cilt 1) 7

4.2 Yazılım (Cilt 2) 7

4.3 Donanım (Cilt 3) 7

4.4 Kontrol (Cilt 4) 8

4.5 Algı ve kestirim (Cilt 5) 8

4.6 Bacaklı robot (Cilt 6) 8

5 Ana Kaynaklar **9**

5.1 Ders videoları 9

5.2 Bu seriyi kullanma şekli 9

Bölüm 1

Bu Seri Nedir, Nasıl Çalışılır

1.1 Hedef: "tam teşekküllü robotik mühendisi" ne demek?

Bir robotik mühendisinden beklenen tek bir beceri değil, birbirine bağlı beş katmanın her birinde *çalışabilir* düzeyde olmak ve en az birinde *derin* olmaktır:

Katman	Soru	Tipik araçlar
Matematik ve mekanik	Robot nerede, nasıl hareket eder, hangi kuvvet gerekir?	Lineer cebir, SE(3), kinematik, Jacobian, dinamik
Kontrol	Robotu istediğim yere nasıl götürürüm, dengede nasıl tutarım?	PID, LQR, MPC, WBC, Lyapunov, trajectory optimization
Algı ve kestirim	Robot nerede olduğunu ve çevresini nasıl bilir?	Kalman/EKF, kamera modeli, SLAM, sensör füzyonu
Yazılım	Bunların hepsini güvenilir, gerçek zamanlı nasıl çalıştırırım?	Linux, Git, modern C++, CMake, ROS 2, test
Donanım	Komut gerçek bir motora, gerçek bir sensörden nasıl gelir?	Motor/sürücü, enkoder, IMU, mikrodenetleyici, haberleşme

Senin hedefin (bacaklı ve humanoid robotlar) bu beş katmanın hepsine *aynı anda* dokunur: bir biped, kontrolün en zor problemlerinden birini (underactuated, temaslı, dengesiz) en kısıtlı donanımla (hafif, güçlü, hızlı aktüatör) ve en gürültülü algıyla (ayak darbeleri, kayma) çözmeyi ister. Bu yüzden serinin ekseni "bacaklı robot" olacak; her cilt diğer alanlara da genel geçer temel verecek.

Dürüst bir uyarı

Hiçbir PDF tek başına kimseyi robotik mühendisi yapmaz. Bu serinin rolü: (1) neyi, hangi sırayla öğreneceğini netleştirmek, (2) her konuyu sayısal örnek ve *çalışan* Python + C++ koduyla anlaşılır kılmak, (3) seni CV'ye konabilecek **projelere** yönlendirmek. Asıl öğrenme, kodları kendin yeniden yazarken, bir şey çalışmadığında hatayı ararken ve projeleri bitirirken olur. Gerçekçi bir süre: haftada 10–15 saatle 12–18 ay.

1.2 Her cildin yapısı

Underactuated rehberindeki formatın aynısı, bir farkla: kodlar artık **Python ve C++ eşit ağırlıkta**.

- **Teori ve türetme:** formülü nereden geldiğiyle birlikte.
- **Sayısal örnek (turuncu kutu):** elle veya hesap makinesiyle takip edilebilecek sayılar.
- **Python kodu** (önce, çünkü hızlı denersin) ve **aynı işi yapan C++ kodu** (Eigen kütüphanesiyle). Her ikisi de gerçekten çalıştırıldı; çıktıları "Çıktı" kutusunda. İki dilin *aynı sayıyı* verdiğini görmek, hem kodun doğruluğunun hem de senin anlamanın testidir.

- **Python/C++ notu (yeşil kutu):** dil tekniği veya tuzak.
- **Alıştırma ve mini proje (mor kutu).**

1.3 Nasıl çalışmalısın

1. **Oku, türet:** formülleri kâğıtta kendin türet. Türetemiyorsan bir önceki konuya dön.
2. **Kodu yeniden yaz, kopyalama:** PDF'teki kodu açık tutup *kendi* dosyanda satır satır yaz. C'de bildiklerinle karşılaştır.
3. **Doğrula:** çıktın PDF'tekiyle aynı mı? Değilse fark nereden geliyor? (Bu adım en çok öğreten adımdır.)
4. **Değiştir ve tahmin et:** bir parametreyi değiştirmeden önce sonucu tahmin et, sonra çalıştır.
5. **Yayınla:** her cildin sonundaki projeyi GitHub'a, kısa bir açıklama ve bir şekil/GIF ile koy. Yüksek lisans başvurusunda ve iş görüşmesinde "ne yaptın?" sorusunun cevabı bu depolar olacak.

1.4 Araç kurulumu (kendi bilgisayarında)

- **İşletim sistemi:** robotikte fiili standart Linux (Ubuntu LTS). Windows kullanıyorsan WSL2 ile Ubuntu kur; ROS 2 ve derleme araçları orada sorunsuz çalışır.
- **Python:** `numpy`, `scipy`, `matplotlib`; ileride `mujoco`, `torch`. Sanal ortam: `python3 -m venv .venv`.
- **C++:** `g++` (veya `clang++`), `cmake`, `Eigen` (Ubuntu'da `sudo apt install libeigen3-dev`). Bu serideki her C++ dosyası tek komutla derlenir: `g++ -std =c++17 -O2 -I /usr/include/eigen3 dosya.cpp -o dosya`.
- **Editör:** VS Code + C/C++ ve Python eklentileri; `clangd` ile otomatik tamamlama.
- **Simülatör:** MuJoCo (Python paketi `pip install mujoco`).

Bu PDF'leri hazırladığım ortamda `pip` ve `apt` dış ağa kapalıydı; bu yüzden MuJoCo, ROS 2, PyTorch ve Pinocchio'yu *çalıştıramadım*. Bu araçlara ait kod parçaları ilgili ciltlerde "çalıştırılmadı" notuyla verilecek; çalıştırılan her şey numpy/scipy ve Eigen ile yazıldı.

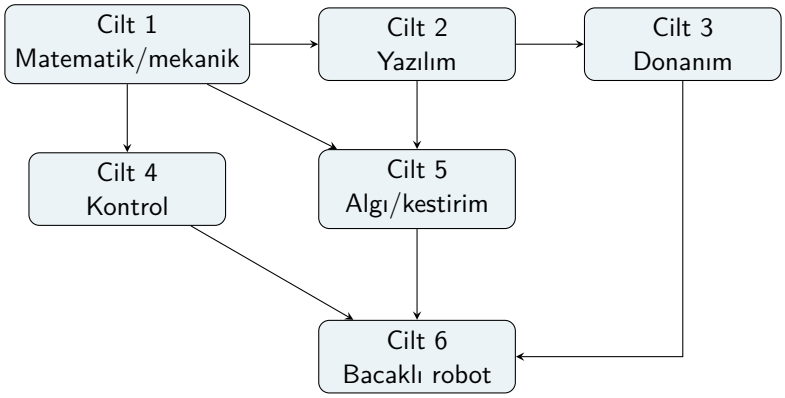
Bölüm 2

Ciltler ve Yol Haritası

2.1 Yedi cilt

Cilt	Başlık	İçerik	Bitirme projesi
0	Yol haritası (bu belge)	Seri planı, proje merdiveni, beceri listesi, kaynaklar	–
1	Matematik ve mekanik temeller	Lineer cebir (SVD, null space), dönmeler (SO(3), kuaterniyon), SE(3), ileri/ters kinematik, Jacobian, statik, rijit cisim dinamiği, RNEA/CRBA, floating base, temas kısıtları, sayısal integrasyon	C++ ve Python'da test edilmiş küçük bir kinematik/dinamik kütüphanesi (bacak modeli)
2	Robot yazılımı	Linux ve kabuk, Git, modern C++ (RAII, şablonlar, akıllı işaretçiler, `std::chrono`), CMake, birim test, gerçek zamanlı döngü tasarımı, ROS 2 (düğüm, topic, servis, parametre, launch, tf2)	Bacak simülasyonu için ROS 2 düğüm mimarisi
3	Donanım ve gömülü kontrol	DC/BLDC motor modelleri, dişli ve şeffaflık, hobi servoları (MG996R) ve sınırları, sürücüler, enkoder/IMU, UART/I2C/CAN, mikrodenetleyicide sabit frekanslı kontrol döngüsü, güç ve güvenlik	Arduino + 2 servo ile 2-DoF bacak: IK + yörünge + ölçüm
4	Kontrol	PID'den başlayıp LQR, MPC, Lyapunov, trajectory optimization, funnel, robust kontrol (Underactuated rehberi bu cildin çekirdeği) + eksik konular eki	Kendi biped modelinde (SB-1) denge/LQR ve ZMP yürüme
5	Algı, kestirim ve SLAM	Kalman/EKF/UKF ve faktör grafları, kamera modeli ve kalibrasyon, epipolar geometri, nokta bulutu ve ICP, görsel/IMU odometri, EKF-SLAM ve poz grafı	Bacaklı robot için taban (floating base) durum kestiricisi
6	Bacaklı robot entegrasyonu	Centroidal dinamik, konveks MPC (MIT Cheetah 3 tarzı), WBC (QP), ayak izi ve yürüyüş zamanlaması, taban kestirimi, RL ile yürüme ve sim-to-real	SB-1'de MPC + WBC ile yürüme, LinkedIn/GitHub'a hazır

2.2 Sıra ve bağımlılıklar



Cilt 1 her şeyin önkoşulu. Cilt 2 (yazılım) ile Cilt 1 *paralel* okunabilir: Cilt 1'deki C++ kodları zaten Cilt 2'nin pratiği. Cilt 4 ve 5 de paralel gidebilir. Cilt 6 hepsini birleştirir.

2.3 Zaman çizelgesi (haftada 10–15 saat)

Ay	Odak	Somut çıktı
1–2	Cilt 1 + Cilt 2'nin ilk yarısı (Linux, Git, C++ temelleri)	GitHub'da kinematik kütüphanesi
3	Cilt 3 + Cilt 2'nin ikinci yarısı (CMake, ROS 2)	Servolu 2-DoF bacak videosu
4–7	Cilt 4 (kontrol)	SB-1'de denge ve basit yürüyüş
6–9	Cilt 5 (algı/kestirim), Cilt 4 ile örtüşerek	Taban kestiricisi + bir kamera kalibrasyonu
9–14	Cilt 6 (entegrasyon)	MPC + WBC yürüyüş, teknik yazı, başvuru portföyü

Bu çizelge bir hedef, sınav değil. Bir konuyu "anladım" saymanın ölçütü: ilgili kodu *bakmadan* yeniden yazabilmek ve sonucunu bir arkadaşına açıklayabilmek.

Bölüm 3

Proje Merdiveni

Projeler, öğrendiğini kalıcı kılan ve başkalarına *gösterebildiğin* tek şey. Aşağıdaki merdiven, elindeki imkânlara göre (bilgisayar, Arduino, iki MG996R servo; 3D yazıcı ve bütçe yok; CAD deneyimi yok) ve hedefine göre (bacaklı/humanoid, yüksek lisans başvurusu) tasarlandı. Her basamak bir öncekinin üzerine kurulur ve her biri tek başına bir GitHub deposu olabilir.

#	Proje	Ne öğretir	Maliyet/araç
P1	**Kinematik kütüphanesi** (Python + C++): 3-DoF bacak için FK, Jacobian, IK, RNEA, birim testler	Cilt 1'in tamamı; C++/Eigen, CMake, test yazma	Sadece bilgisayar
P2	**Masaüstü 2-DoF bacak**: 2 MG996R, Arduino, karton/kontrplak/alüminyum profil gövde	IK'nın gerçek dünyada çalışması; servo gecikmesi, ölü bölge, akım sınırı; seri port haberleşmesi	Elindeki parçalar + basit malzeme
P3	**ROS 2 mimarisi**: simülasyondaki bacak için "durum yayınlayıcı / kontrolcü / yörünge üreteci" düğümleri	Topic/servis tasarımı, zamanlama, launch dosyaları	Bilgisayar (Ubuntu/WSL2)
P4	**SB-1 denge**: kendi yazdığın biped modelinde (MuJoCo) dik duruşta LQR, itmeye karşı tepki	Doğrusallaştırma (`mjd_transitionFD`), LQR, kazanç ayarı	Bilgisayar
P5	**ZMP yürüyüşü**: ön izlemeli kontrol + IK ile SB-1'de yürüme	LIPM, ZMP, ayak yörüngeleri	Bilgisayar
P6	**Taban durum kestiricisi**: IMU + bacak kinematiği EKF (simülasyon verisiyle)	EKF, temas tespiti, NEES ile doğrulama	Bilgisayar
P7	**MPC + WBC yürüyüşü**: centroidal konveks MPC + QP tabanlı WBC	Cilt 4–6'nın birleşimi; gerçek zamanlı optimizasyon	Bilgisayar
P8	**RL ile yürüme** (isteğe bağlı): domain randomization ile politika	PPO, ödül tasarımı, sim-to-real riskleri	GPU'lu bilgisayar faydalı

3.1 Seri boyunca kullanılacak robot: SB-1

GO2B'ye artık erişimin olmadığı için seri, **kendi yazacağın** bir biped modeli üzerine kuruludur. Adı bu belgede **SB-1** ("seri bipedi"); istediğin adı verebilirsin. Avantajı: modelin her parametresini sen seçtiğin ve her dosyasını sen yazdığın için portföyde tamamen sana ait bir proje olur ve hiçbir dış dosyaya bağımlı değilsin.

- **Yapı:** gövde + iki bacak, bacak başına 5–6 eklem (kalça yaw/roll/pitch, diz pitch, ayak bileği pitch/roll); toplam kütle 2–3 kg, uyluk ve baldır 0.2 m (Cilt 1'deki bacak örnekleriyle aynı

ölçüler, bu yüzden oradaki kodlar doğrudan SB-1'in bacak kodu olur).

- **Cilt 1:** bacak kinematiği, Jacobian, IK ve dinamiği Python + C++ kütüphanesi olarak (`robkit`).
- **Cilt 2:** SB-1'in MJCF (MuJoCo XML) dosyasını sıfırdan yazmak; kütle/atalet değerlerini Cilt 1'deki formüllerle hesaplamak; ROS 2 düğümleriyle sarmalamak.
- **Cilt 4–6:** denge, ZMP yürüyüşü, MPC + WBC ve RL hep bu model üzerinde.

Karşılaştırma için açık kaynak modeller (MuJoCo Menagerie deposunda, ücretsiz): `robotis_op3` (servo motorlu küçük humanoid; hobi servolarıyla çalışan senin donanım dünyana en yakını), `toddlerbot_2xc`, `berkeley_humanoid`, `booster_t1`, `unitree_h1`/`unitree_g1` ve `agility_cassie`. Kendi algoritmanı SB-1'de çalıştırdıktan sonra bunlardan birinde de denemek ("aynı WBC, farklı robot"), portföyde güçlü bir genelleme kanıtı olur.

3.2 P2 hakkında: MG996R ile neyi yapabilir, neyi yapamazsın?

MG996R bir *hobi pozisyon servosu*dur: içinde DC motor, dişli kutusu, potansiyometre ve kendi kapalı çevrim kontrolcüsü vardır; dışarıdan sadece PWM ile *hedef açı* verirsin. Bu yüzden:

- **Yapabilirsin:** IK ile ayak ucunu bir yörüngede gezdirme, statik duruş, yavaş adım hareketleri, kinematik doğrulama (ayak ucu ölçülen konumla tahmin edilen konum karşılaştırması).
- **Yapamazsın (düzgün):** tork kontrolü, WBC, dinamik yürüyüş. Servo açı geri bildirimi vermez (potansiyometre sinyalini dışarı almak için servoyu açıp kablo lehimlemek gerekir), torku ölçemezsin, iç kontrolcünün kazancını değiştiremezsin. Datasheet değerleri (yaklaşık 9–11 kg·cm durma torku, besleme gerilimine bağlı) "durma" torkudur; sürekli kullanılabilir tork bunun çok altındadır.

Bu sınırlamaları Cilt 3'te sayısal olarak modelleyip neden gerçek bacaklı robotların (MIT Cheetah, Unitree vb.) düşük dişli oranlı, akım kontrollü BLDC "proprioceptive" aktüatörler kullandığını göreceğiz. P2'nin amacı mükemmel bir robot değil: *model ile gerçeğin farkını kendi gözünle görmek* ve bunu belgelemek.

3.3 Portföy ve başvuru için

Yüksek lisans başvurusunda (ve iş görüşmesinde) güçlü olan şey: **az ama derin ve belgelenmiş** proje. Her proje deposunda:

1. Bir cümlelik ne yaptığı + bir GIF veya şekil (en üstte).
2. Nasıl çalıştırılacağı (tek komut).
3. Ne doğrulandı: sayılar, karşılaştırma, test sonuçları ("EKF'nin NEES'i %95 bandında"; "Python ve C++ RNEA 10^{-12} içinde aynı").
4. Neyin çalışmadığı ve neden (dürüstlük, akademik değerlendirmede artı puandır).

Görüşmede ve başvuruda en çok işe yarayan şey, *her satırını açıklayabildiğin* kod. Bu yüzden serideki projelerin hepsi sıfırdan, kendi modelin üzerinde yapılıyor (bkz. bir sonraki bölüm).

Bölüm 4

Beceri Kontrol Listesi

Her maddeyi 0 (bilmiyorum), 1 (okudum/anladım), 2 (kodunu yazdım), 3 (bir projede kullandım ve başkasına anlatabilirim) diye puanla. Hedef: tüm maddelerde en az 2, kendi uzmanlık alanında (bacaklı robot kontrolü) 3.

4.1 Matematik ve mekanik (Cilt 1)

- ☐ SVD ile bir matrisin rankını, null uzayını, koşul sayısını bulup yorumlamak
- ☐ Rotasyon matrisi, Euler açıları, eksen-açı ve kuaterniyon arasında dönüşüm; gimbal lock'u göstermek
- ☐ Homojen dönüşüm ve twist ile ileri kinematik (DH veya PoE)
- ☐ Geometrik Jacobian'ı türetmek ve sonlu farkla doğrulamak; $\boldsymbol{\tau} = \mathbf{J}^T\mathbf{F}$
- ☐ Sayısal ters kinematik (damped least squares), null uzayında ikincil görev
- ☐ Lagrange ile 2-link dinamiği; RNEA ile ters dinamik; $\mathbf{M}(\mathbf{q})$, $\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$
- ☐ Floating base, kütle merkezi, centroidal momentum; temas kısıtlı dinamiği KKT ile çözmek

4.2 Yazılım (Cilt 2)

- ☐ Linux kabuğunda dosya, süreç, izin, ortam değişkeni; `ssh`
- ☐ Git: branch, merge, rebase, pull request; anlamlı commit geçmişi
- ☐ Modern C++: sınıflar, RAII, `std::vector`, `std::unique_ptr`, şablonlar, lambda, `const`-doğruluk
- ☐ CMake ile kütüphane + çalıştırılabilir + test hedefi
- ☐ Sabit frekanslı kontrol döngüsü; jitter ölçmek
- ☐ ROS 2: düğüm, topic, servis, parametre, launch, tf2, `ros2 bag`

4.3 Donanım (Cilt 3)

- ☐ DC motor elektriksel/mekanik modeli; tork sabiti, dişli oranı, yansıyan atalet
- ☐ Akım (tork), hız ve konum kontrol döngüleri; PWM
- ☐ Enkoder çözünürlüğü, IMU gürültü/kayma (bias) modeli
- ☐ UART, I2C, SPI, CAN farkları; bir sensörü mikrodenetleyiciye bağlayıp okumak
- ☐ Güç bütçesi, akım sınırı, acil durdurma

4.4 Kontrol (Cilt 4)

- ☐ PID ayarı ve sınırları (doyma, anti-windup, türev filtresi)
- ☐ LQR, Riccati; kazanç/faz payı; gecikme payı
- ☐ Lyapunov ile kararlılık ve çekim bölgesi
- ☐ Trajectory optimization (collocation, iLQR), MPC (QP)
- ☐ Limit cycle, Poincaré haritası, hibrid sistemler

4.5 Algı ve kestirim (Cilt 5)

- ☐ KF/EKF/UKF ve tutarlılık testi (NEES/NIS)
- ☐ Pinhole kamera modeli, iç/dış parametreler, distorsiyon, kalibrasyon
- ☐ Epipolar geometri, PnP; ICP ile nokta bulutu hizalama
- ☐ Poz grafı / faktör grafı optimizasyonu

4.6 Bacaklı robot (Cilt 6)

- ☐ LIPM, ZMP, DCM/capture point
- ☐ Centroidal konveks MPC; sürtünme konisi
- ☐ QP tabanlı WBC (görev öncelikleri, kısıtlar)
- ☐ Taban durum kestirimi (IMU + kinematik)
- ☐ RL ile yürüme ve domain randomization

Bölüm 5

Ana Kaynaklar

Seri bu kaynakların *yerine* değil, onlara giden yolu kısaltmak için yazılıyor. Her alan için önce bir "çekirdek" kaynak, sonra derinleşme.

Alan	Çekirdek kaynak	Derinleşme
Kinematik/dinamik	K. Lynch & F. Park, *Modern Robotics* (ücretsiz PDF, Coursera dersi, videolar)	R. Featherstone, *Rigid Body Dynamics Algorithms*; Siciliano vd., *Robotics: Modelling, Planning and Control*
Bacaklı robot dinamiği	ETH Zürih *Robot Dynamics* ders notları (M. Hutter grubu)	Wieber, Tedrake, Kuindersma, "Modeling and Control of Legged Robots" (Springer Handbook of Robotics bölümü)
Kontrol	R. Tedrake, *Underactuated Robotics* (+ bu serinin Cilt 4'ü)	J. Rawlings vd., *Model Predictive Control*; K. Åström & R. Murray, *Feedback Systems* (ücretsiz)
Humanoid	S. Kajita vd., *Introduction to Humanoid Robotics*	Di Carlo vd. 2018, "Dynamic Locomotion in the MIT Cheetah 3 Through Convex MPC"
Kestirim	S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*	T. Barfoot, *State Estimation for Robotics*
Görme	R. Szeliski, *Computer Vision: Algorithms and Applications* (ücretsiz)	Hartley & Zisserman, *Multiple View Geometry*
C++	B. Stroustrup, *A Tour of C++*; `learncpp.com`	S. Meyers, *Effective Modern C++*
ROS 2	Resmi belgeler ve eğitimler (`docs.ros.org`)	Örnek projeler: `ros2_control`
Öğrenme	Sutton & Barto, *Reinforcement Learning* (ücretsiz)	Isaac Lab / legged_gym örnekleri; Diffusion Policy (Chi vd. 2023)
Simülasyon	MuJoCo belgeleri	Pinocchio (hızlı dinamik, türevler), Drake

5.1 Ders videoları

- **Modern Robotics** (Northwestern, Coursera): Cilt 1 ile birebir örtüşür; her video 5–10 dakika.
- **MIT 6.832/6.8210 Underactuated Robotics** (Tedrake): Cilt 4.
- **ETH Robot Dynamics**: Cilt 1'in floating base kısmı ve Cilt 6.

5.2 Bu seriyi kullanma şekli

Her cildin başında "bu cilt, şu kaynağın şu bölümleriyle birlikte okunmalı" diye bir eşleme olacak. Bir kavramı serideki anlatımdan anlamadıysan, aynı konuyu çekirdek kaynaktan oku ve geri dön: aynı fikri iki farklı anlatımdan görmek çoğu zaman kilidi açar.